COMP3358 Assignment 3: JPoker 24-Game

Distributed and Parallel Computing

1. Program Organization

The system consists of a server and a Swing GUI client, communicating via two transport layers:

RMI (request/response): login, register, logout, getStats, getLeaderboard.
JMS (event-driven): game-joining via Queue, game-playing/results via Topic.

File	Description
JPoker24GameServer.java	Main server. RMI service + JMS queue listener + topic publisher + game state machine + JDBC.
JPoker24Game.java	Main client. Swing GUI + RMI caller + JMS queue producer + topic subscriber.
GameService.java	RMI remote interface (login, register, logout, getStats, getLeaderboard).
GameMessage.java	Serializable payload for JMS ObjectMessage (JOIN, ANSWER, PLAYER_JOINED, GAME_START, GAME_R
PlayerStats.java	Serializable stats object returned by RMI (wins, played, avgWinTimeMs, rank).
Expr24.java	24-game expression evaluator with exact rational arithmetic. Validates card usage and result.

JMS Resources (GlassFish 5):
Connection Factory: jms/JPoker24GameConnectionFactory
Queue: jms/JPoker24GameQueue (client-to-server: JOIN, ANSWER)
Topic: jms/JPoker24GameTopic (server-to-clients: PLAYER_JOINED, GAME_START, GAME_RESULT)

Database: MySQL database *c3358*, tables UserInfo, OnlineUser, GameStats. Leaderboard updates use explicit transactions (setAutoCommit(false), batch upsert, commit).

Game State Machine (server-side):
IDLE: waiting for first JOIN.
JOINING: collecting players. Starts game when 4 players join, or when 2+ players and 10 seconds elapsed since first join.
PLAYING: 4 cards dealt (guaranteed solvable, all distinct values 1-13). First player to submit a correct expression equaling 24 wins. Expression validated with exact rational arithmetic (supports +, -, *, /, parentheses, fractions in intermediates).
After game ends: updates stats transactionally, broadcasts result, returns to IDLE (or JOINING if late joiners queued).

Only one game runs at a time, as specified.

2. How to Compile

Prerequisites: Java 8 (e.g. Amazon Corretto 8), GlassFish 5.1.0, MySQL with database c3358.

```
export JAVA_HOME=/path/to/java8
export PATH=$JAVA_HOME/bin:$PATH
CP="$HOME/glassfish5/glassfish/lib/gf-client.jar:/path/to/mysql-connector-j-9.3.0.jar:."
cd src
javac -cp "$CP" *.java
```

3. How to Run

Step 1: Start GlassFish and create JMS resources (one-time setup):

```
~/glassfish5/bin/asadmin start-domain
~/glassfish5/bin/asadmin create-jms-resource --restype javax.jms.ConnectionFactory
jms/JPoker24GameConnectionFactory
```

```
~/glassfish5/bin/asadmin create-jms-resource --restype javax.jms.Queue --property
Name=JPoker24GameQueue jms/JPoker24GameQueue
~/glassfish5/bin/asadmin create-jms-resource --restype javax.jms.Topic --property
Name=JPoker24GameTopic jms/JPoker24GameTopic
```

Step 2: Create the GameStats table:

```
mysql -u c3358 -pc3358PASS c3358 -e "CREATE TABLE IF NOT EXISTS GameStats (
  username VARCHAR(64) PRIMARY KEY, wins INT DEFAULT 0,
  played INT DEFAULT 0, total_win_time_ms BIGINT DEFAULT 0,
  FOREIGN KEY (username) REFERENCES UserInfo(username));"
```

Step 3: Start server:

```
java -cp "$CP" -Djava.security.policy=security.policy JPoker24GameServer
```

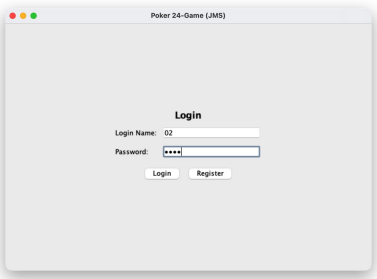
Step 4: Start client(s) (one per player):

```
java -cp "$CP" JPoker24Game
```

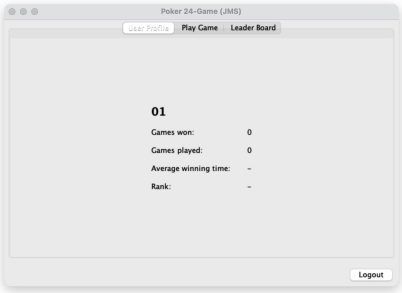
Convenience scripts: ./run_server.sh, ./run_client.sh

4. Screenshots

Login screen



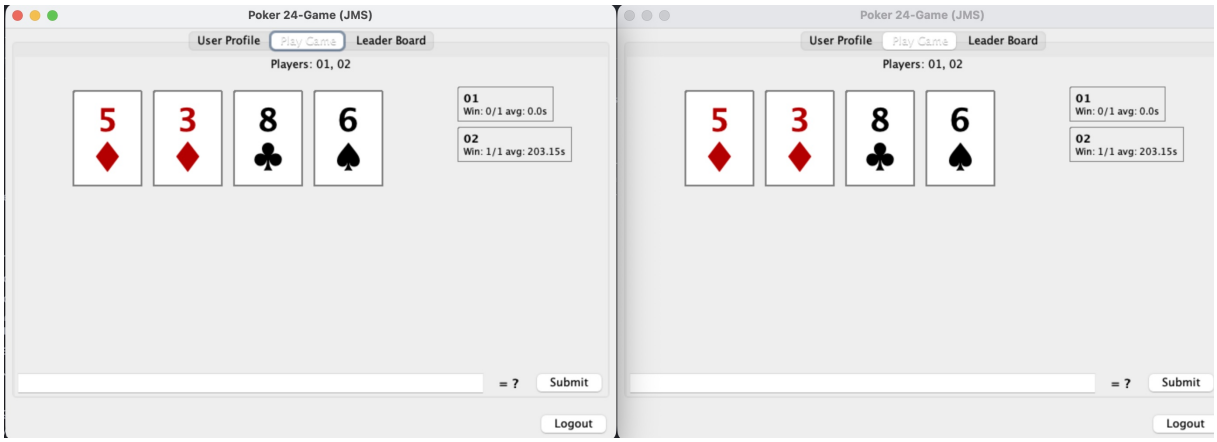
User Profile (initial)



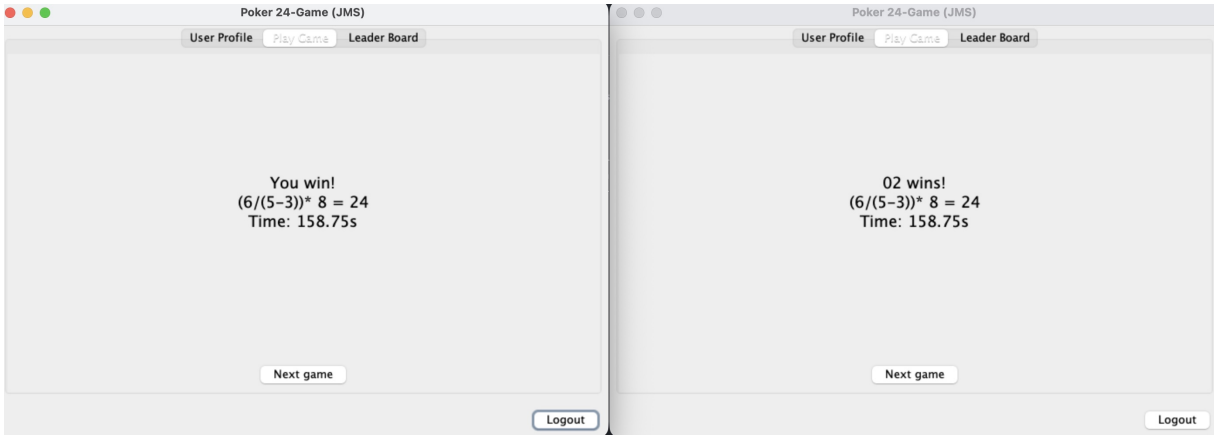
Initial: New Game button

Game-joining: waiting

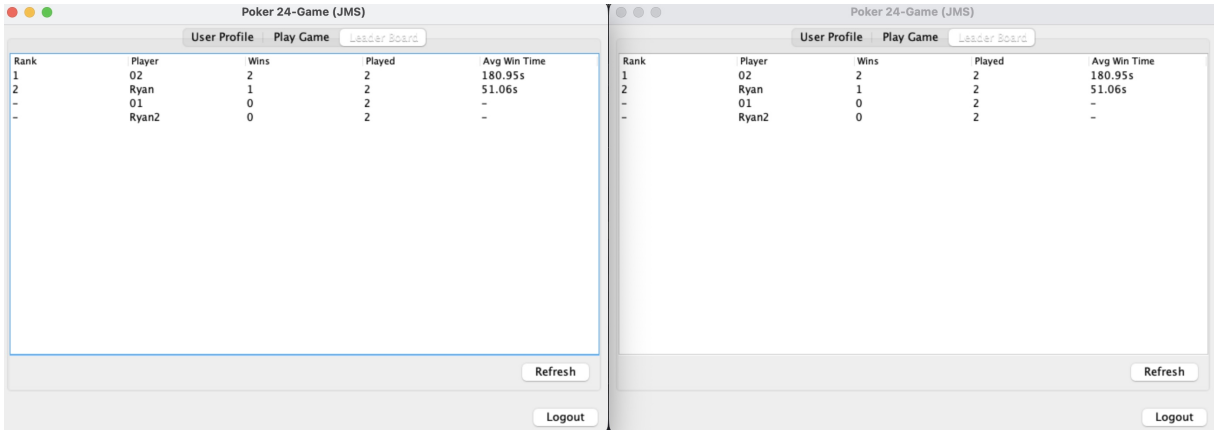
Game-playing: cards dealt, both clients (2 players)



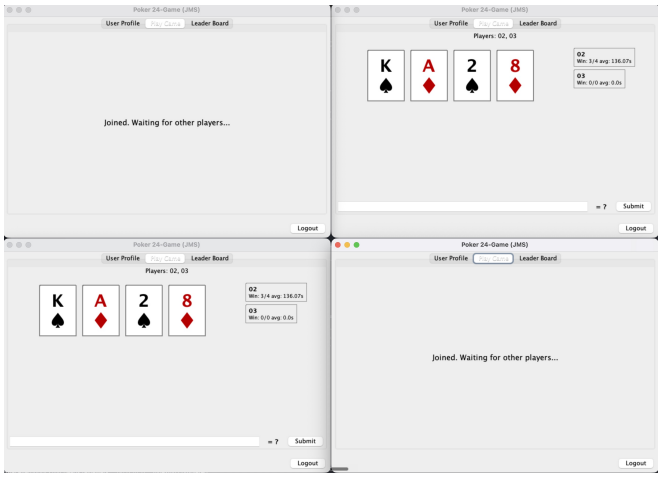
Game-over: winner announced on both clients



Leaderboard: ranked by wins, both clients



Multi-stage: joining and playing simultaneously



4-player game: all four clients in playing stage